

FeKAN: Efficient Kolmogorov-Arnold Networks Accelerator Using FeFET-based CAM and LUT

Xuliang Yu, Yu Qian, Xunzhao Yin, Cheng Zhuo, and Liang Zhao*
College of Information Science and Electronic Engineering, Zhejiang University

Abstract—Kolmogorov-Arnold networks (KANs) have emerged as a promising alternative to MLP due to their adaptive learning capabilities for complex dependencies through B-spline basis activations (BBA). However, existing in-memory accelerators optimized for MLP-based DNNs are primarily designed for vector-matrix multiplication (VMM), making them inefficient for the dynamic and recursive B-spline interpolation (BSI) operations required by KANs. In this work, we propose FeKAN, an FeFET-based architecture designed to accelerate BBA operations. First, we develop a software-hardware co-optimized framework for mapping B-spline basis functions (BBF), leveraging a two-stage design space exploration (DSE) algorithm in combination with FeFET-based Look-Up Tables (LUT) and Content-Addressable Memory (CAM). This framework translated dynamic BSI operations into static codebook lookups, achieving a balanced trade-off between memory and computational efficiency. Second, we propose compress-sparsity-column (CSC) based encoding for B-spline basis function and grouped-computation strategy for memory and energy reduction. Third, we propose a grouped-pipeline optimization strategy to mitigate data dependencies, significantly enhancing computation efficiency. Experimental results demonstrate that FeKAN achieves up to $150.68\times$ and $4664\times$ higher throughput and up to $606.87\times$ and $11196\times$ greater energy efficiency over Intel Xeon Silver 4310 CPU and NVIDIA A6000 GPU, respectively.

I. INTRODUCTION

The rapid advancement of artificial intelligence (AI) is profoundly transforming the world, with technologies like large language models (LLMs), autonomous driving and digital twins significantly redefining industries and enhancing user experience. Among the diverse deep learning architectures, MLP-based deep neural networks (DNNs [1], [2]) stand out for their ability to capture local features and extract high-level abstract information from sequences through their multi-layered structures. Despite their remarkable achievements, DNNs face significant limitations including fixed activation functions and large memory footprint, hindering their performance in high-dimensional data modeling and complex problem-solving. Recently, KAN [3] has emerged as a promising architecture for efficient data fitting through B-spline interpolation with trainable activation functions.

Research on accelerating KAN networks remains limited. In contrast, most efforts to accelerate MLP-based DNNs have primarily focused on sparse computation optimization and computational structure management to enhance computational efficiency. Sparse computation optimization involves pruning weakly-correlated neurons [4]–[6] and employing low-precision quantization [7], [8] to reduce redundant computation and memory footprint. Further improvements include designing sparse matrix multiplication units [9], [10] that compute only non-zero elements. In contrast, computational structure management focuses on balancing computational and memory resources through data-flow control mechanisms [11]–[13], multi-granular hardware parallelization [5], [14], and memory optimization

techniques [15], [16]. Despite their impressive performance, these approaches are not directly applicable to KAN. This limitation arises as B-spline interpolation requires frequent retrieval of embeddings from off-chip DRAM to perform recursive computations, which are not well-suited for conventional DNN-based accelerators.

Most recently, compute-in-memory (CIM) has emerged as a promising non-von Neumann architectural approach to accelerate MLP-based DNNs with enhanced parallelism and energy efficiency. These architectures embed computations directly within the memory arrays, reducing the need for frequent off-chip data access and movement, thereby circumventing the memory bottleneck and improving both the computational and energy efficiency. CIM variants include SRAM-based digital CIM (DCIM) [17], [18], analog CIM (ACIM) [19], [20] and emerging non-volatile memory (eNVM) ACIM [21]–[25], among others. Currently, SRAM-DCIM and ACIM demonstrate higher accuracy than eNVM-ACIM in specific tasks. However, the larger area, higher standby power and inherent volatility of SRAM cells significantly limit their application in energy-constrained edge devices. In contrast, eNVMs such as RRAM, MRAM, PCM and FeFET can encode data into varying conductance states, enabling VMM operations to be mapped and processed in the analog domain through voltage/current transformations. For instance, ISSAC [26], PRIME [27], FELIX [28] and iCELIA [29] support energy-efficient VMM acceleration for DNNs using crossbar arrays of eNVM cells.

Although eNVM-ACIM designs offer substantial potential for accelerating DNNs, the dynamic order-associated interpolation operations within KAN are not well aligned with the static VMM nature of eNVM-ACIM. This misalignment poses significant challenges in terms of write endurance, cache bandwidth and read/write energy efficiency for KAN acceleration with eNVM. To illustrate this misalignment, Fig. 1(a) outlines the major challenge: **(1) BSI operations act as the bottleneck of KAN on existing hardwares.** The k^{th} -order BBF require recursive and node-by-node interpolation, resulting in frequent memory access for intermediate results and excessive computation delays. Profiling reveals that BBA operations account for $91.31\% \sim 96.6\%$ of KAN inference time across various GPU platforms. In BBA computation, BSI contributes 65.6% of the total latency. To address the challenge of irregular and dynamic BSI operations, we transform the BBF into a static codebook and embed them into FeFET memory, leveraging the local support property within B-spline functions. Such practices transform the above challenge into two detailed challenges (Fig. 1 (b-c)): **(2) Multi-scale BSI curve mapping brings large memory footprint.** Different layers in KAN exhibit diverse data distributions, requiring the B-spline codebook mapping to adapt to multi-scale input ranges, sampling resolutions and quantization bit-widths for more precise approximation. Although the codebook memory footprint decreases with the reduced sampling points, the model precision also degrades significantly in extreme cases, necessitating a careful trade-off between the two. **(3) Non-local sparsity of B-spline functions causes redundant calculations and**

*Liang Zhao is the corresponding author (e-mail: lzhao2020@zju.edu.cn).

This work was supported by the National Science and Technology Major Project of the Ministry of Science and Technology of China (Grant No. 2024YFE0201100), the National Natural Science Foundation of China (Grant No. 62474162), and the Natural Science Foundation of Zhejiang Province (Grant No. LD25F040003).

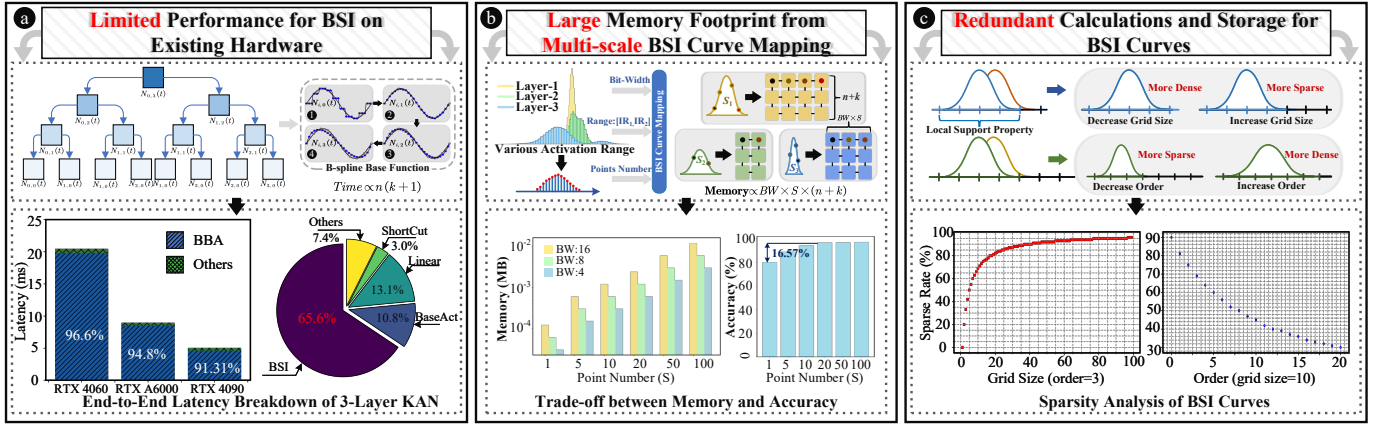


Fig. 1: Challenges in KAN inference: (a) Limited performance for BSI operations on existing hardware, (b) Large memory footprint from multi-scale BSI curve mapping and (c) Redundant calculations and storage for BSI curves.

storage. The BBF features non-local sparsity coupled with the order and control point number, which introduces computation redundancy in basis function weighting. When the order is fixed, the sparsity within the k^{th} -order base function increases from zero to nearly 100% as the number of control points increases. Conversely, with a fixed number of control points, increasing order allows the k^{th} -order base function to cover a broader range without sacrificing accuracy.

To resolve these challenges, we propose FeKAN, an FeFET-based architecture for KAN acceleration, which implements a software-hardware co-optimized B-spline mapping scheme to achieve memory-friendly and energy-efficient BBA computation. The key contributions of this work are summarized as follows:

- We propose a software-hardware co-optimized BBF mapping framework that combines the two-stage DSE algorithm and LUT-CAM mapping scheme, effectively balancing memory and computational efficiency.
- We propose a CSC-based encoding for BBF along with grouped computation strategy, effectively reducing the memory occupancy and energy consumption.
- we propose a grouped-pipeline optimization strategy to mitigate data dependencies within BSI computations, significantly enhancing computation efficiency.
- Experimental results confirm that FeKAN offers significant advantages over CPUs and GPUs for KAN acceleration, especially in terms of end-to-end latency, throughput and energy efficiency.

II. BACKGROUND

Kolmogorov-Arnold Theorem: For arbitrary continuous multi-variable function $f(x_1, x_2, \dots, x_n)$ on a bounded domain, there always exists a finite composition of continuous univariate functions ϕ_{ij} and ψ_i such that $f: [0, 1]^n \rightarrow \mathbb{R}$ can be represented as a weighted sum of these functions, that is:

$$f(x_1, x_2, \dots, x_n) = \sum_{i=1}^{2n+1} \psi_i \left(\sum_{j=1}^n (\phi_{ij}(x_j)) \right) \quad (1)$$

Just as the universal approximation theorem underpins the development of artificial neural networks, this Kolmogorov-Arnold theorem serves as the theoretical foundation of the KAN algorithm. Moreover, unlike the universal approximation theorem, this theorem focuses on transforming high-dimensional functions into compositions of univariate functions to simplify the computation, providing significant advantages for the design of hierarchical network structures.

B-spline Interpolation (BSI): BSI is widely used in computational science as a flexible and efficient tool for approximating functions. BSI curves are generated by recursive linear combination of control

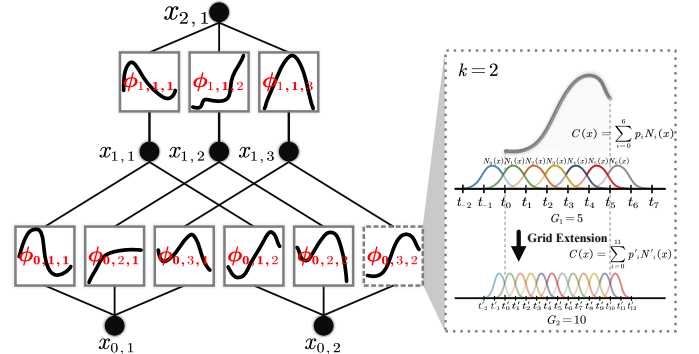


Fig. 2: KAN architecture and the mathematical principles of BSI.

point set and BBF, leveraging their local support property. This process typically involves two computational phases: BBF generation and BSI curve construction.

• **B-spline Basis Function (BBF) Generation.** For a k -degree BBF with n control points, the k -degree basis functions are constructed iteratively, starting from zero-base function. Here, k determines the smoothness and polynomial degree of the basis functions, while the knot vector $\{t_0, t_1, \dots, t_{n+k}\}$ defines the support interval and shape of the basis function curve. Specifically, the basis functions can be obtained through the following recursive procedure:

$$N_{i,0}(x) = \begin{cases} 1, & \text{if } t_i \leq x < t_{i+1} \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

$$N_{i,k}(x) = \frac{x - t_i}{t_{i+k} - t_i} N_{i,k-1}(x) + \frac{t_{i+k+1} - x}{t_{i+k+1} - t_{i+1}} N_{i+1,k-1}(x)$$

where $N_{i,k}(x)$ represents the k -degree BBF associated with the i^{th} control point for the input x .

• **B-spline Curve Construction.** Once the BBF are obtained, they can be linearly combined with the control points set $\mathbf{P} = \{p_1, \dots, p_n\} \in \mathbb{R}^{n \times 1}$ to generate the B-spline interpolation results:

$$C(x_j) = \sum_{i=1}^n p_i N_{i,k}(x_j) \longleftrightarrow C = A \cdot P \quad (3)$$

where $\{x_j\}_{j=1}^r$ represent the interpolation points and $C(x_j) \subset C \in \mathbb{R}^{r \times 1}$ denotes the BSI values corresponding to $\{x_j\}_{j=1}^r$. $A \in \mathbb{R}^{r \times n}$ is the interpolation coefficient matrix, indicating the contribution of the i^{th} control point to the j^{th} interpolation point.

KAN Architecture: Inspired by the Kolmogorov-Arnold theorem, KAN incorporates a hierarchical activation mechanism by combining the SiLU($\cdot$) and B-Spline($\cdot$) functions. SiLU($\cdot$) serves as the base

activation (BaseAct), providing a smooth initial state, while the B-spline activation enhances adaptability with its flexible shape and local adjustment capability. As shown in Fig. 2, M -layer KAN can be recursively expanded into a composition of functions:

$$\text{KAN}(x) = (\psi_{M-1} \circ \psi_{L-2} \circ \dots \circ \psi_0)(x), \psi = \{\phi_{i,j}\} \quad (4)$$

where $\text{spline}(x)$ can be parameterized as a linear combination of BBF with respect to the control points set $\mathbf{P}$ in Eq. 3. The weight $\mathbf{w}_b$, $\mathbf{w}_s$ and the control points $\mathbf{p}_i$ s can be trained for different tasks to adjust the activation function's response over various input intervals.

FeFET and Its In-Memory Computing Applications: compared to other eNVMS [30]–[32], FeFET devices offer some unique advantages for CIM including the voltage-driven mechanisms, high ON/OFF ratio and very low OFF current associated with the three-terminal FET structure [33]. As a typical use case, FeFET-based CAM accelerates parallel associative search operations throughout the entire memory array, demonstrating promising speed and energy efficiency that is difficult to achieve with other two-terminal eNVMS [34], [35]. In addition, FeFET crossbar arrays can store weight matrices and perform parallel VMM to accelerate neural networks [36] and accelerate combinatorial optimization [37]–[39].

III. ALGORITHM

A. Two-stage Design Space Exploration

Translating the dynamic BBF into a static codebook offers an optimal space-time trade-off. Specifically, we store the input activations and their corresponding B-spline values in memory as dictionary pairs. For each query, the key is matched in the codebook, and the corresponding value is returned as BBF. Although this method effectively improves computational efficiency and reduces memory access overhead, the pursuit of higher inference performance and memory efficiency remains constrained by several factors, including the input range, number of sampling points and quantization precision. These factors are elaborated as follows:

- The distribution of input activations across different layers of KAN is affected by the network structure and the specific transduction tasks, making it difficult to establish a universal and widely applicable codebook that can approximate the true data distribution. This, in turn, impacts the inference accuracy of KAN based on the codebook.
- The number of sampling points and input range jointly affect the resolution of the codebook. Too few sampling points result in insufficient discretization accuracy in the input space, making it impossible to map the input data correctly and causing B-spline mapping distortion. Conversely, over-sampling increases memory and computational overhead during the query-key matching.
- Model quantization leads to flattening and overlapping effects in the values of BBF. Thus, neighboring B-spline data points lose the ability to distinguish differences in the original data, resulting in the same quantized value after mapping. This fails to accurately capture the nonlinear activation characteristics of the BBF, which in turn degrades the model accuracy.

To mitigate these risks, we propose a two-stage design space exploration algorithm (see Algorithm 1) to optimize the trade-off between model performance and memory load, which is influenced by parameters such as the input range, number of sampling points and quantization bit-width. The goal is to construct an efficient codebook that simultaneously achieves high fitting accuracy and low memory load. In the first stage, we update the knot vector $\{t_i\}_{i=0}^{n+k}$ using the predefined control points number n , spline degree k , and grid intervals gr . Based on the input range ir and the number of sampling

Algorithm 1 Two-stage Design Space Exploration (DSE) Algorithm

Input: Base-model, Transduction-Task, ϵ , Input Range Set $\{IR\}$, Sampling Point Set $\{S\}$, Bit-Width Set $\{BW\}$, Search Grid $\{\tau_{coarse}, \tau_{fine}\}$, BSI-degree k , knot-vector range $\{gr\}$ and control points n .
Output: Best Parameter Set $\{IR_{best}, S_{best}, BW_{best}\}$, Shared BSI Codebook.

```

1: Initialize Best_Score  $\leftarrow 0$ 
2: for layer in layers do
3:    $IR_{coarse}, S_{coarse}, BW_{coarse} \leftarrow \{None, None, None\}$ 
4:   for  $\{bw, s, ir\}$  in  $\{BW_{coarse}, S_{coarse}, IR_{coarse}\}$  step  $\tau_{coarse}$  do
5:     knot_vector  $\leftarrow \text{Generate\_Knot\_Vector}(gr, k, n, bw)$ 
6:      $\{Keys\} \leftarrow \text{Input\_Sampling}(ir, s, bw)$ 
7:     BSI-Codebook  $\leftarrow \text{Compute\_Bspline}(\text{knot\_vector}, Keys, k, bw)$ 

8:   Model  $\leftarrow \text{Update}(\text{Base-model}, \text{BSI-Codebook}, \text{knot\_vector})$ 
9:   Accuracy  $\leftarrow \text{Evaluate}(\text{Model}, \text{Transduction-Task})$ 
10:  if  $abs(\text{Accuracy} - \text{Base-accuracy}) \leq \epsilon$  then
11:    Score  $\leftarrow \text{Fitness}(\text{Accuracy}, bw, s, k, n)$ 
12:    if Score > Best_Score then
13:       $IR_{coarse}, S_{coarse}, BW_{coarse} \leftarrow \{s, bw, ir\}$ 
14:      Best_Score  $\leftarrow$  Score
15:    end if
16:  end if
17: end for
18: end for
19: Repeat Lines 2-18 with  $\{IR_{coarse}, S_{coarse}, BW_{coarse}\}$  for finer search.
    Fitness( $\cdot$ ) :=  $\alpha \times \text{Accuracy} + \beta \times (bw \times s \times k \times n)$ 

```

points s , we generate a candidate input query set $\mathcal{Q}$. Then, we acquire the shared codebook using the BBF computation method (Eq. 2), which takes the candidate set $\mathcal{Q}$ and the knot vector $\{t_i\}_{i=0}^{n+k}$ as inputs. Afterward, we refine the parameter search space by evaluating the model task and applying a fitness function reward. This aims to determine the optimal set of parameters that balance both codebook storage requirements and model accuracy. Finally, a two-stage finer search further optimizes the parameters near the solution obtained from the previous coarse search, using a finer step size τ_{fine} .

B. GQA-LUT-based SiLU Approximation

The BaseAct (e.g. $\text{SiLU}(\cdot)$) is introduced as residual nonlinear term in KAN to enable the learning of more complex decision boundaries and data distributions. However, the computation of nonlinear functions involves complex data-flow transformations, which are not well-suited to the CIM-based integer computation architecture. Hence, an eNVM-friendly method for nonlinear computation is highly desirable.

Due to the linear inseparability of nonlinear functions, directly estimating scaling factors and quantizing nonlinear functions can distort the original data distribution, leading to a degradation of model accuracy and learning capability. Piecewise linear approximation (pwl) decomposes nonlinear functions into simpler data segments within specific intervals [40], making it a promising candidate for accelerating B-splines. Based on the $\text{pwl}(\cdot)$ method, we can convert the $\text{SiLU}(\cdot)$ calculation into a series of linear mapping expressions represented by N entries $\text{pwl}_i(x)_{i=0}^{N-1}$, as shown below:

$$\text{pwl}(x) = \begin{cases} k_0x + b_0, & \text{if } x < p_0, \\ k_1x + b_1, & \text{if } p_0 \leq x < p_1, \\ \vdots & \\ k_{N-1}x + b_{N-1}, & \text{if } x \geq p_{N-2}. \end{cases} \quad (5)$$

in which the breakpoints $p_{i=0}^{N-1}$ determine the interval location of each entry, while the slope k_i and intercept b_i define the mapping from the input to $\text{pwl}(x)$.

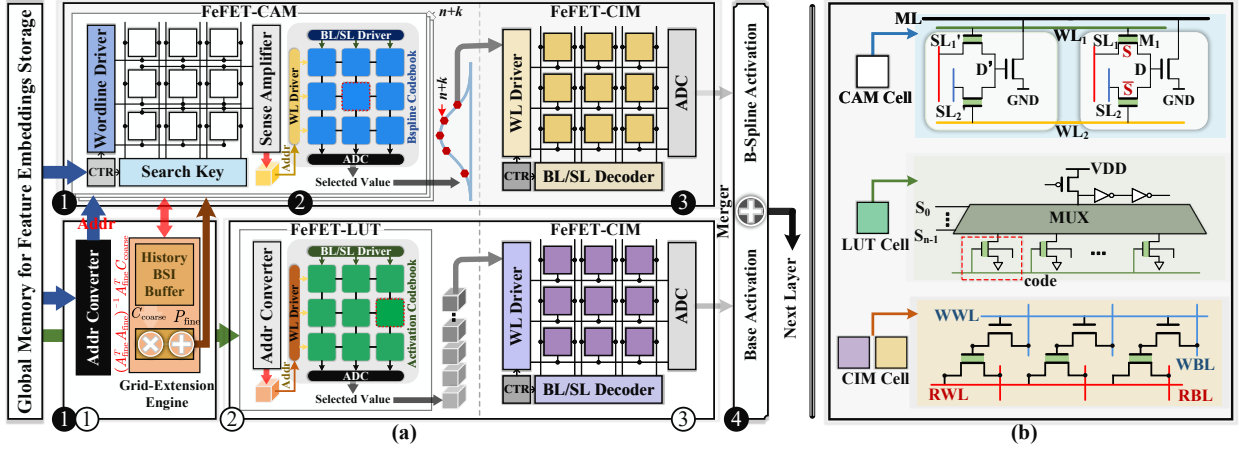


Fig. 3: (a) FeKAN accelerator architecture (BSI Dataflow: ①–④, BaseAct Dataflow: ①–④); (b) CAM/LUT/CIM cell structure.

To improve the nonlinear computation efficiency of SiLU on the FeFET-based CIM architecture, we adopt the genetic piecewise linear approximation method from Genetic Quantization-Aware LUT-Approximation (GQA-LUT) [41]. The breakpoint set $\mathcal{O} = \{P_1, P_1, \dots, P_{N_p-1}\}$ is randomly initialized within the interval $[R_n, R_p]$. During each evolution phase of the genetic algorithm, the approximation result is evaluated by calculating the mean square error between the nonlinear function and the piecewise linear approximation. The crossover operation combines the breakpoints of two individuals to generate new candidates, while the mutation operation adjusts the breakpoint locations to explore new solution spaces. A 3-size tournament selection is applied to choose the individuals with higher fitness for the next generation. The final output of the best slope, intercept and breakpoint candidates can be used to construct the piecewise asymptotic scheme for SiLU, effectively approximating the non-asymptotic function.

IV. ARCHITECTURE

A. Overall Architecture

Fig. 3 shows the overall architecture of FeKAN, which integrates FeFET-based CAM, LUT and CIM computation macros to support both BSI and BaseAct operations. To better elucidate the two dataflows within the architecture, we illustrate this by using the BSI computation as an example: ① Feature embeddings are first loaded on-chip. Then, the BBF mapping is implemented in either the CAM or LUT computation unit based on the configured mode. The CAM adopts a query-key matching approach to generate position-selection signals, while the LUT employs a content-address mapping list for address conversion. ② Upon receiving the matched signal from the CAM or LUT, the B-spline codebook group, with a depth of $n+k$, reads out the corresponding B-spline mapping values and column indices of the inputs, facilitating subsequent sparse computations. ③ The BBF results are weighted by the control point matrix $\mathbb{P}$, which is embedded in the FeFET-CIM array, to generate the final BSI results. ④ Finally, the BSI and BaseAct results are merged by the Merger engine, after which the generated activation information is forwarded to the next layer. The BaseAct follows the same computation manner as BSI. Besides, we incorporate an on-chip grid-extension engine to support fine-grained BSI operations.

B. FeFET-based CAM and LUT

Following the two-stage DSE algorithm, we obtain a shared static codebook for the input queries $\mathcal{Q}_{i=0}^M$ across all layers. This codebook can then be embedded into memory for BBF mapping of feature embeddings. In this work, we used FeFET-based CAM and LUT

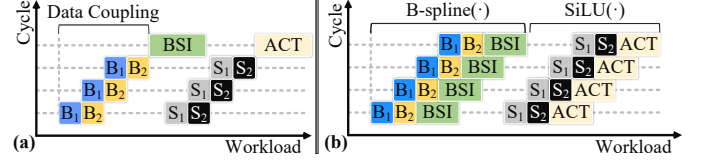


Fig. 4: (a) Raw computation stack and (b) pipeline optimization.

to implement the query-key primitive in both BBF and BaseAct, as depicted on the left top of Fig. 3(b).

We employed a 2FeFET-1T array [35] for CAM. Each row shares the same wordlines (WLs) and matchline (ML), and each column shares the same searchlines (SLs) within the array. The MLs are connected to a sense amplifier for output sensing. The schematic of the 2FeFET-1T CAM cell is illustrated on the top of Fig. 3(b), where each cell consists of a pair of FeFETs and an nMOS discharge transistor. Before search operation, each bit of the word in the codebook is encoded and written into the cell as the V_{th} of FeFETs. During search, each bit in the search key is encoded as voltages applied to SLs. The voltage of node D will be high/low enough to turn on/off the nMOS for the mismatch/match case, respectively. Upon a search, only the row where all cells are matched remains a high voltage of ML, which is sensed by the sense amplifier for further data fetch. For FeFET-LUT, we use a multiplexer formed by n -bit inputs to decode the input address into row/column select signals [42], which are then used to access and read specific FeFET cells.

C. CSC-based Encoding and Grouped-Computation

To address the non-local sparsity introduced by BBF, we propose a CSC-based encoding and grouped-computation strategy for compact BSI computation. As illustrated in Fig. 5(a), the raw B-spline codebook is represented as a structured 2D grid, where the column vectors correspond to the sampled points along the input distribution, and the row vectors represent the interpolation points derived from various BBFs. During encoding, arbitrary feature point x within row-vector can be sequentially extracted for zero-detection and unfolding. For each non-zero element, a column address is assigned to identify its exact location in the B-spline matrix, and the corresponding $\{\text{column index, B-spline}(x)\}$ pair is generated. This pair is then written into the address and content buffers, providing efficient input support for subsequent computations. Based on this encoding scheme, the codebook memory footprint is reduced by a factor of $\frac{n+k}{k'}$, where k' is less than $k+1$.

Furthermore, we introduce a grouped-computation strategy, which partitions the B-spline set $\in \mathbb{R}^{F \times (n+k)}$ into F distinct groups,

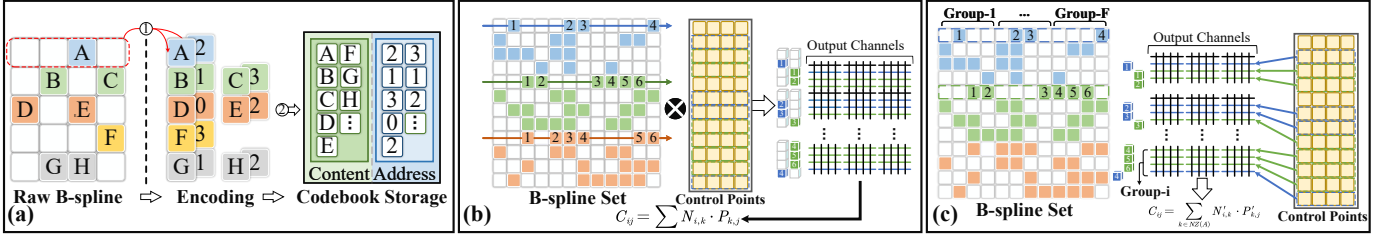


Fig. 5: (a) CSC-based sparse encoding, (b) conventional computation and (c) optimized sparse grouped-computation.

where F denotes feature dimension. Each group corresponds to the BBF associated with a specific feature point. Within each group, local weighted calculations are carried out using the interpolation coefficient matrix $N_{i,k}$ in conjunction with the control points $P_{k,j}$. By leveraging the encoding method for BBFs discussed earlier, we can efficiently compute only the non-zero entries, denoted as $NZ(A)$ using Eq. 6, thereby further optimizing computational efficiency.

$$C_{i,j} = \sum_k N_{i,k} \cdot P_{k,j} = \sum_{(k) \in NZ(A)} N'_{i,k} \cdot P'_{k,j}, \quad (6)$$

$$NZ(A) = \{k : N_{i,k} \neq 0 \text{ and } P_{k,j} \neq 0\}$$

D. Pipeline Optimization

In the BBA computation process, both the B-spline activation function and the basic nonlinear activation functions have parallelization potential. However, due to the data coupling in large array computation tasks, the parallelization capability has not been fully realized. For example, the generation of B-spline curves depends on the preparation and computation of BBFs in phases B1 and B2 (see Fig. 4 (a)). In this mode, the output of each phase must wait for all preceding tasks to complete, and this dependency causes resource idling, significantly limiting computational efficiency.

To address this issue, we propose an optimization method based on grouped pipelining. By tiling the control point matrix of the raw large array, we decompose the complex matrix operations into fine-grained single-element basis function weighting computations, achieving higher parallelism. At the same time, by grouping the basis functions and processing them layer by layer in a pipelined manner, we effectively reduce phase dependencies and decrease the waiting time between tasks. This method significantly improves overall computational efficiency and provides a new approach for parallel optimization in long feature sequence inference tasks.

E. Grid Extension Engine

Considering the local support and piecewise polynomial structure of BBFs, the B-spline function can be refined through grid extension. This allows it to more precisely capture complex features and subtle variations. Suppose P_{coarse} and A_{coarse} represent the original control point matrix and interpolation coefficient matrix, these matrices will become P_{fine} and A_{fine} after grid refinement. To ensure alignment between the refined spline and the original, we can formulate a least-squares objective as follows:

$$\min_{P_{\text{fine}}} \|A_{\text{fine}} \cdot P_{\text{fine}} - C_{\text{coarse}}\|_2^2 \quad (7)$$

where the solution to Eq. 7 can be expressed as follows:

$$P_{\text{fine}} = (A_{\text{fine}}^T A_{\text{fine}})^{-1} A_{\text{fine}}^T C_{\text{coarse}} \quad (8)$$

To achieve off-line grid refinement, an on-chip digital multiplier-accumulator circuit is used together with the history BSI buffer storing C_{coarse} to update the control point weights P_{fine} .

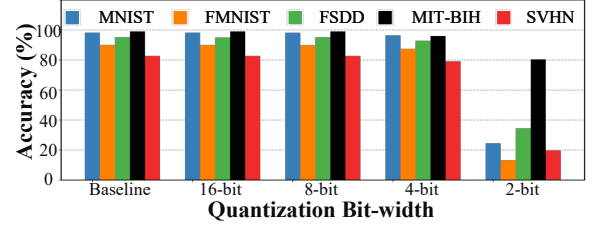


Fig. 6: Impact of Quantization Bit-width on Model Accuracy.

TABLE I: Parameter Configurations for KAN

Model	order k	control points n	layer configurations
KAN-I	3	10	[784, 256, 64, 10]
KAN-II	5	10	[300, 64, 5]
KAN-III	5	10	[3072, 512, 128, 10]

V. EVALUATION

A. Evaluation Methodology

We evaluated the performance of FeKAN using three different KAN models across open-source benchmark tasks spanning three modalities: image classification (MNIST, FMNIST, SVHN), speech recognition (FSDD), and electrocardiogram (ECG) sequence analysis (MIT-BIH). The specific model parameters are listed in Table I, where MNIST, FMNIST, and FSDD are evaluated using KAN-I. We employed a two-stage design space exploration algorithm with $\tau_{\text{coarse}} = 1$ and $\tau_{\text{fine}} = 0.1$ to identify the optimal trade-offs between accuracy and memory usage across different tasks. This process involved sampling the best points for bit-width quantization and input range sets.

The performances of FeFET-CAM and LUT were validated using Cadence Virtuoso based on a 45nm FeFET model calibrated with experimental data [35] [42]. Power consumption and latency of the designs were estimated through Spectre simulations. For the weighted operations of BBA, we simulated the FeFET-CIM performances in NeuroSim V1.4 [43] at the 32nm process node, which implements the BSI and residual activation function SiLU($\cdot$).

Additionally, to demonstrate the acceleration effectiveness of the BBA within FeKAN, we evaluated power consumption and latency on Intel Xeon Silver 4310 CPUs, and Nvidia RTX 4060, 4090, and A6000 GPUs. Latency for both GPU and CPU was measured using CUDA Events and Python Timers, while power consumption was monitored using Nvidia-smi and PyRAPL.

B. Algorithm Performance

Impact of Quantization Bit-Width. We first evaluated how varying the quantization precision from 2 to 16 bits impacts the performance of the KAN model across different benchmark tasks, as shown in Fig. 6. The KAN model demonstrates strong quantization tolerance across various tasks. Specifically, when the quantization precision is reduced to 8 bits, the model maintains performance comparable to the baseline. However, when the quantization precision drops below 4 bits, a sharp decline in performance is observed. This degradation

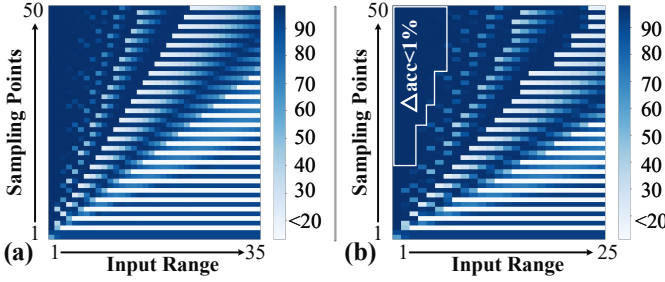


Fig. 7: Search results visualization: (a) BW = 32-bit, (b) BW = 8-bit.

is attributed to the inability of low-bit quantization to fully capture the data distribution of the dataset.

DSE Analysis. We visualized the accuracy distribution of the KAN-I model under quantization precisions of 16 and 8 bits, across different input ranges and sampling point numbers. The accuracy of the KAN-I model exhibits a block-like variation in the input range, sampling points parameter space. Specifically, when the number of sampling points is fixed and the input range is large, or when the input range is fixed and the number of sampling points is small, most real activation values are mapped to the same value range, resulting in the loss of fine-grained information and, consequently, a decrease in model performance. Therefore, increasing the number of sampling points within a given input range effectively maintains model accuracy. The parameter space with $\Delta_{acc} < 1\%$, indicating an inference accuracy loss relative to the baseline, is highlighted. Based on this analysis, the optimal parameter space for this task is identified as an input range of $\{-1, +1\}$ and 10 sampling points.

C. Hardware Performance

End-to-End SpeedUp and Energy Efficiency. Fig. 8 presents the end-to-end speedup and energy efficiency improvements of FeKAN compared to CPUs across different computation platforms. FeKAN-CAM achieves up to $2.74\times$ the speedup of FeKAN-LUT. FeKAN-CAM achieves a maximum speedup of $3620.41\times$ and $2303.32\times$ over the RTX 4090 and A6000, respectively, across five tasks, while FeKAN-LUT can achieve a maximum speedup of $54943\times$ over the CPU. Two key factors contribute to these results: first, the B-spline interpolation mapping based on CAM and LUT effectively bypasses dynamic recursive interpolation calculations, shortening the computational path length associated with order coupling; second, grouped pipelined operations decouple the B-spline curve construction from the basis function generation process, enhancing parallelism.

Regarding energy efficiency, FeKAN-LUT achieves the highest performance improvement, with up to $606.87\times$ better energy efficiency compared to the CPU, while FeKAN-CAM achieves a $304.17\times$ improvement in energy efficiency. In comparison to A6000 and RTX 4060 GPUs, FeKAN-CAM demonstrates energy efficiency improvements of $207.19/5612.01\times$ and $9.16/154.78\times$, respectively. The exceptional energy efficiency of FeKAN can be attributed to its CIM architecture which alleviates the von-Neumann bottleneck, as well as computational optimizations leveraging sparse encoding.

Ablation Study. Fig. 9 shows the latency and energy breakdown of three KAN models (KAN-I, KAN-II, and KAN-III) on FeKAN when sparse optimization and pipelining optimization strategies are applied. The experimental results indicate that the KAN-I and KAN-II models significantly reduce the B-spline CIM computation time through the pipelining optimization strategy, with a compression factor ranging from $6.86\times$ to $7.43\times$. This demonstrates that the pipelining optimization strategy successfully avoids phase-dependent

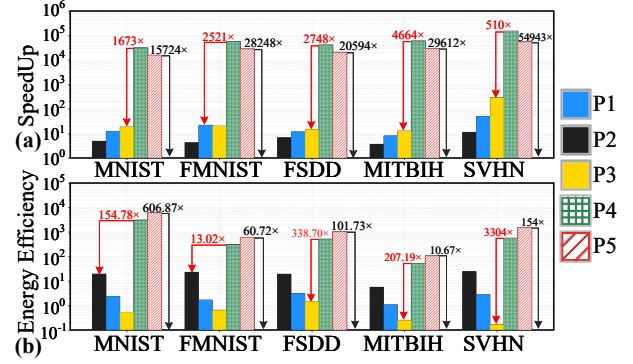


Fig. 8: Comparison of Latency and Energy Efficiency: FeKAN vs. RTX 4060/4090/A6000 GPUs (P1-P3) and Intel Xeon Silver 4310 CPU, with P4 and P5 Representing FeKAN-CAM and FeKAN-LUT.

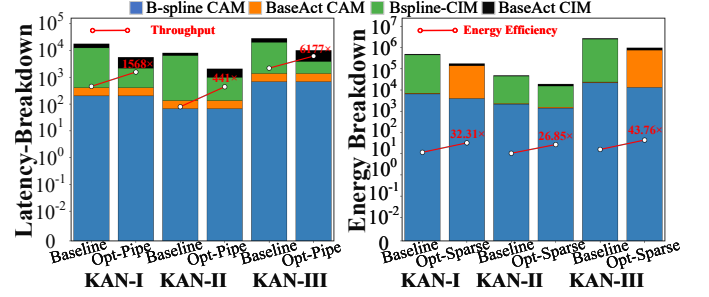


Fig. 9: Latency and energy breakdown of FeKAN with sparse and pipeline optimizations versus baseline.

data waiting by stacking data along the time dimension, thereby improving overall computational efficiency.

Furthermore, with the application of the grouped pipelining optimization strategy, KAN-III achieves a throughput of 6177 GOPS, representing a $2.82\times$ improvement over the baseline model. Regarding energy breakdown, the sparse optimization strategy significantly reduces the energy consumption of CAM and CIM during query-key matching and BBF weighting. Specifically, KAN-III and KAN-I achieve energy efficiencies of 43.76 TOPS/W and 32.31 TOPS/W, respectively. This performance improvement can be attributed to the sparse encoding and grouped-computation strategy. By reducing redundant weighting calculations and storage requirements in the BBFs, this strategy greatly reduces computational complexity, leading to lower power consumption and higher energy efficiency.

VI. CONCLUSION

In this work, we propose FeKAN, an FeFET-based in-memory accelerator designed to address the unique challenges of accelerating Kolmogorov-Arnold Networks (KANs) through efficient and memory-friendly B-spline basis activation computations. FeKAN incorporates a software-hardware co-optimized framework that combines a two-stage design space exploration (DSE) algorithm with a LUT-CAM mapping scheme, effectively balancing memory overhead and interpolation efficiency. To further reduce memory usage and redundant computation, we introduce a CSC-based encoding for BBF compression and a grouped-computation strategy for compact and efficient processing. Additionally, we propose a grouped-pipeline optimization strategy to improve computational efficiency and throughput. Experimental results demonstrate FeKAN's superiority over conventional CPUs and GPUs, achieving significant reductions in end-to-end latency while delivering substantial gains in throughput and energy efficiency for KAN acceleration.

REFERENCES

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [2] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.
- [3] Z. Liu, Y. Wang, S. Vaidya *et al.*, "KAN: Kolmogorov-Arnold Networks," 2024, arXiv:2404.19756.
- [4] S. Han, X. Liu, H. Mao *et al.*, "EIE: Efficient inference engine on compressed deep neural network," in *Proc. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 243–254.
- [5] S. Zhang, Z. Du, L. Zhang *et al.*, "Cambricon-X: An Accelerator for Sparse Neural Networks," in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2016, pp. 1–12.
- [6] J.-F. Zhang, C.-E. Lee, C. Liu *et al.*, "SNAP: An efficient sparse neural acceleration processor for unstructured sparse deep neural network inference," *IEEE J. Solid-State Circuits (JSSC)*, vol. 56, no. 2, pp. 636–647, 2021.
- [7] K. Wang, Z. Liu, Y. Lin *et al.*, "HAQ: Hardware-Aware Automated Quantization With Mixed Precision," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 8604–8612.
- [8] D. Zhang, J. Yang, D. Ye *et al.*, "Lq-Nets: Learned Quantization for Highly Accurate and Compact Deep Neural Networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 365–382.
- [9] N. Srivastava, H. Jin, J. Liu *et al.*, "MatRaptor: A Sparse-Sparse Matrix Multiplication Accelerator Based on Row-Wise Product," in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2020, pp. 766–780.
- [10] S. Pal, J. Beaumont, D.-H. Park *et al.*, "OuterSPACE: An Outer Product Based Sparse Matrix Multiplication Accelerator," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2018, pp. 724–736.
- [11] Y.-H. Chen, T. Krishna, J. Emer *et al.*, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [12] A. Parashar, M. Rhu, A. Mukkara *et al.*, "SCNN: An accelerator for compressed-sparse convolutional neural networks," *SIGARCH Comput. Archit. News*, vol. 45, no. 2, pp. 27–40, 2017.
- [13] W. Lu, G. Yan, J. Li *et al.*, "FlexFlow: A flexible dataflow accelerator architecture for convolutional neural networks," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, 2017, pp. 553–564.
- [14] Y. Chen, T. Luo, S. Liu *et al.*, "DaDianNao: A Machine-Learning Super-computer," in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2014, pp. 609–622.
- [15] J. Li, S. Jiang, S. Gong *et al.*, "SqueezeFlow: A sparse CNN accelerator exploiting concise convolution rules," *IEEE Trans. Comput. (TC)*, vol. 68, no. 11, pp. 1663–1677, 2019.
- [16] V. Sze, Y.-H. Chen, T.-J. Yang *et al.*, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [17] M.-E. Shih, S.-W. Hsieh, P.-Y. Tsai *et al.*, "NVE: A 3nm 23.2 TOPS/W 12b-digital-cim-based neural engine for high-resolution visual-quality enhancement on smart devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 67, 2024, pp. 360–362.
- [18] H. Fujiwara, H. Mori, W.-C. Zhao *et al.*, "A 5-nm 254-TOPS/W 221-TOPS/mm² fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 65, 2022, pp. 1–3.
- [19] X. Si, Y.-N. Tu, W.-H. Huang *et al.*, "A 28nm 64Kb 6T SRAM computing-in-memory macro with 8b MAC operation for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2020, pp. 246–248.
- [20] J.-W. Su, X. Si, Y.-C. Chou *et al.*, "A 28nm 64Kb inference-training two-way transpose multibit 6T SRAM compute-in-memory macro for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2020, pp. 240–242.
- [21] C. Matsui, K. Toprasertpong, S. Takagi *et al.*, "FeFET local multiply and global accumulate voltage-sensing computation-in-memory circuit design for neuromorphic computing," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 3, pp. 468–479, 2024.
- [22] B. Li, P. Gu, Y. Shan *et al.*, "RRAM-based analog approximate computing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst. (TCAD)*, vol. 34, no. 12, pp. 1905–1917, 2015.
- [23] S. Angizi, Z. He, A. Awad *et al.*, "MRIMA: An MRAM-based in-memory accelerator," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst. (TCAD)*, vol. 39, no. 5, pp. 1123–1136, 2020.
- [24] V. Joshi, M. Le Gallo, S. Haefeli *et al.*, "Accurate deep neural network inference using computational phase-change memory," *Nat. Commun.*, vol. 11, no. 1, p. 2473, 2020.
- [25] X. Yin, X. Chen, M. Niemier *et al.*, "Ferroelectric FETs-based non-volatile logic-in-memory circuits," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, pp. 159–172, 2019.
- [26] A. Shafiee, A. Nag, N. Muralimanohar *et al.*, "ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *Proc. ACM/IEEE Annu. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 14–26.
- [27] P. Chi, S. Li, C. Xu *et al.*, "PRIME: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory," in *Proc. ACM/IEEE Annu. Int. Symp. Comput. Archit. (ISCA)*, 2016, pp. 27–39.
- [28] T. Soliman, N. Laleni, T. Kirchner *et al.*, "FELIX: A ferroelectric FET based low power mixed-signal in-memory architecture for DNN acceleration," *ACM Trans. Embed. Comput. Syst. (TECS)*, vol. 21, no. 6, pp. 84:1–84:25, 2022.
- [29] H. Yan, H. R. Cherian, E. C. Ahn *et al.*, "iCELIA: A full-stack framework for STT-MRAM-based deep learning acceleration," *IEEE Trans. Parallel Distrib. Syst. (TPDS)*, vol. 31, no. 2, pp. 408–422, 2020.
- [30] Y. Qian, L. Zhao, F. Meng *et al.*, "Enhancing convnets with convfifo: A crossbar pim architecture based on kernel-stationary first-in-first-out dataflow," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, no. 01, pp. 1640–1651, 2024.
- [31] D. Ielmini and H.-S. P. Wong, "In-memory computing with resistive switching devices," *Nat. Electron.*, vol. 1, pp. 333–343, 2018.
- [32] C. Zhuo, Z. Yang, K. Ni *et al.*, "Design of ultra-compact content addressable memory exploiting 1T-1MTJ cell," *Trans. Comput.-Aided Design Integr. Circuits Syst. (TCAD)*, 2022.
- [33] K. Ni, X. Yin, A. F. Laguna *et al.*, "Ferroelectric ternary content-addressable memory for one-shot learning," *Nat. Electron.*, vol. 2, no. 11, pp. 521–529, 2019.
- [34] X. Yin, Y. Qian, M. Imani *et al.*, "Ferroelectric ternary content addressable memories for energy-efficient associative search," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst. (TCAD)*, vol. 42, no. 4, pp. 1099–1112, 2022.
- [35] Y. Qian, Z. Fan, H. Wang *et al.*, "Energy-aware designs of ferroelectric ternary content addressable memory," in *Proc. Des. Autom. Test Eur. Conf. Exhib. (DATE)*, 2021, pp. 1090–1095.
- [36] Z. Yang, Q. Huang, Y. Qian *et al.*, "Energy efficient dual designs of FeFET-based analog in-memory computing with inherent shift-add capability," in *Proc. 61st ACM/IEEE Des. Autom. Conf. (DAC)*, 2024, pp. 1–6.
- [37] X. Yin, Y. Qian, A. Vardar *et al.*, "Ferroelectric compute-in-memory annealer for combinatorial optimization problems," *Nat. Commun.*, vol. 15, no. 1, p. 2419, 2024.
- [38] Y. Qian, K. Ni, T. Kampfe *et al.*, "C-Nash: A novel ferroelectric computing-in-memory architecture for solving mixed strategy Nash equilibrium," in *Proc. 61st ACM/IEEE Des. Autom. Conf. (DAC)*, 2024, pp. 1–6.
- [39] Y. Qian, Z. Yang, K. Ni *et al.*, "HyCiM: A hybrid computing-in-memory qubo solver for general combinatorial optimization problems with inequality constraints," in *Proc. 61st ACM/IEEE Des. Autom. Conf. (DAC)*, 2024, pp. 1–6.
- [40] J. Yu, J. Park, S. Park *et al.*, "NN-LUT: Neural approximation of non-linear operations for efficient transformer inference," in *Proc. 59th ACM/IEEE Des. Autom. Conf.*, 2022.
- [41] P. Dong, Y. Tan, D. Zhang *et al.*, "Genetic quantization-aware approximation for non-linear operations in transformers," in *Proc. 61st ACM/IEEE Des. Autom. Conf.*, 2024.
- [42] Y. Xu, Z. Zhao, Y. Xiao *et al.*, "Ferroelectric FET-based context-switching fpga enabling dynamic reconfiguration for adaptive deep learning machines," *Sci. Adv.*, vol. 10, no. 3, 2024.
- [43] X. Peng, S. Huang, Y. Luo *et al.*, "DNN+NeuroSim: An end-to-end benchmarking framework for compute-in-memory accelerators with versatile device technologies," in *IEEE International Electron Devices Meeting (IEDM)*, 2019.